

Applied Machine Learning

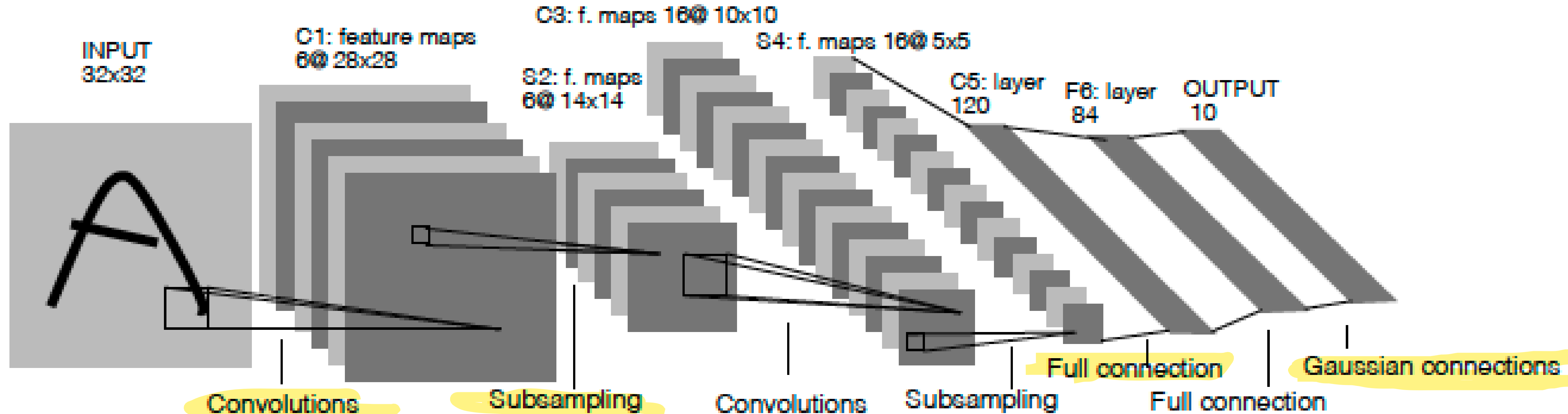
Convolutional Neural Network

Selpi

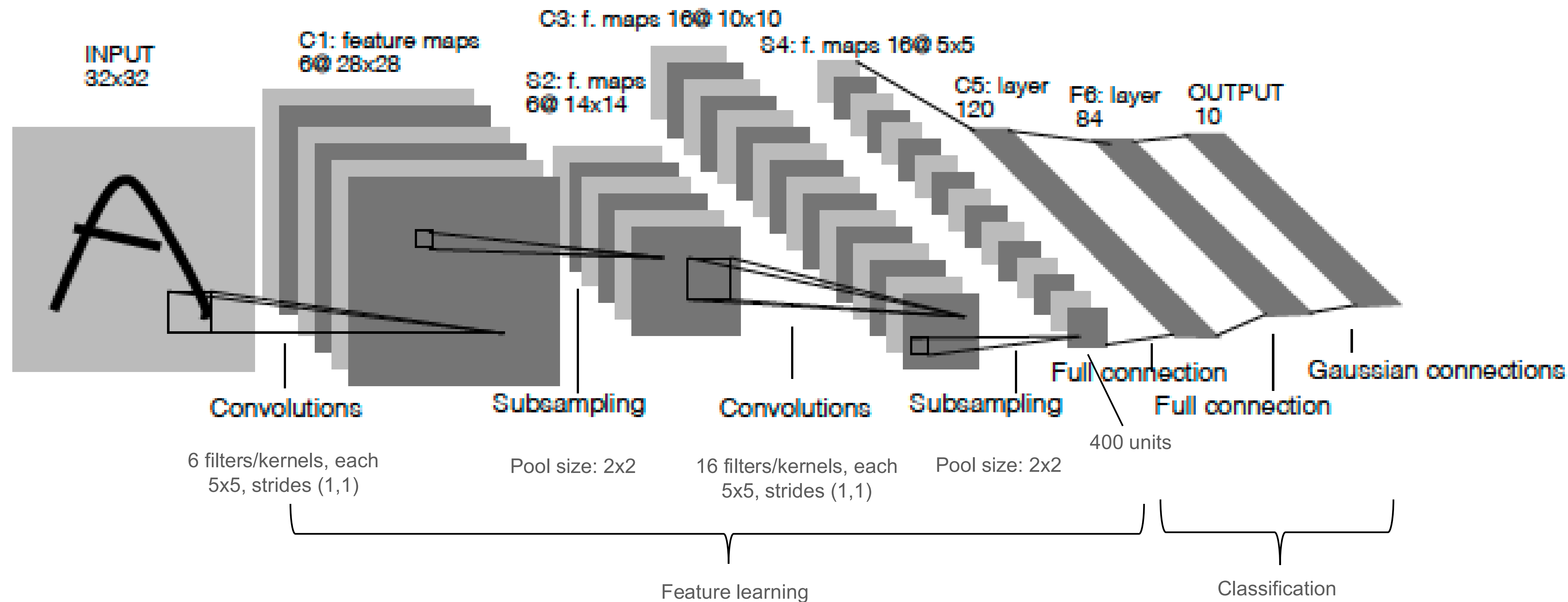
Building blocks of CNNs -

- Convolutional filters to spot patterns
- Pooling to summarise regions
- Fully connected/dense layers, like a regular NN
- To allow training of deep networks, consider residual connections and batch normalisations

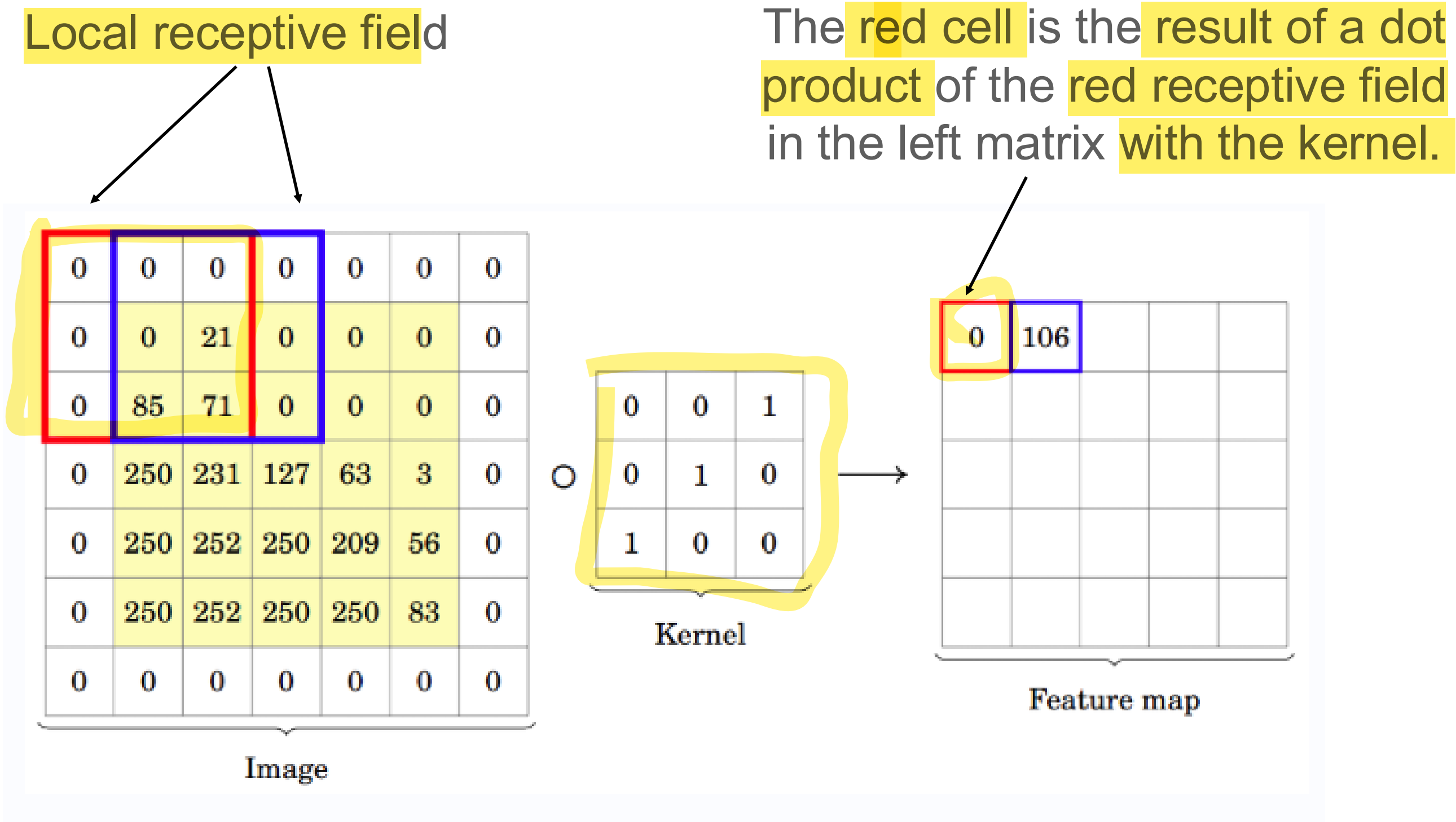
LeNet – an example of CNN architecture



LeNet – an example of CNN architecture



Convolutional filter



Apply activation function after conv layer

For ReLU (the most commonly used for CNN)

- $f(x) = \max(0, x)$
- What is the result?

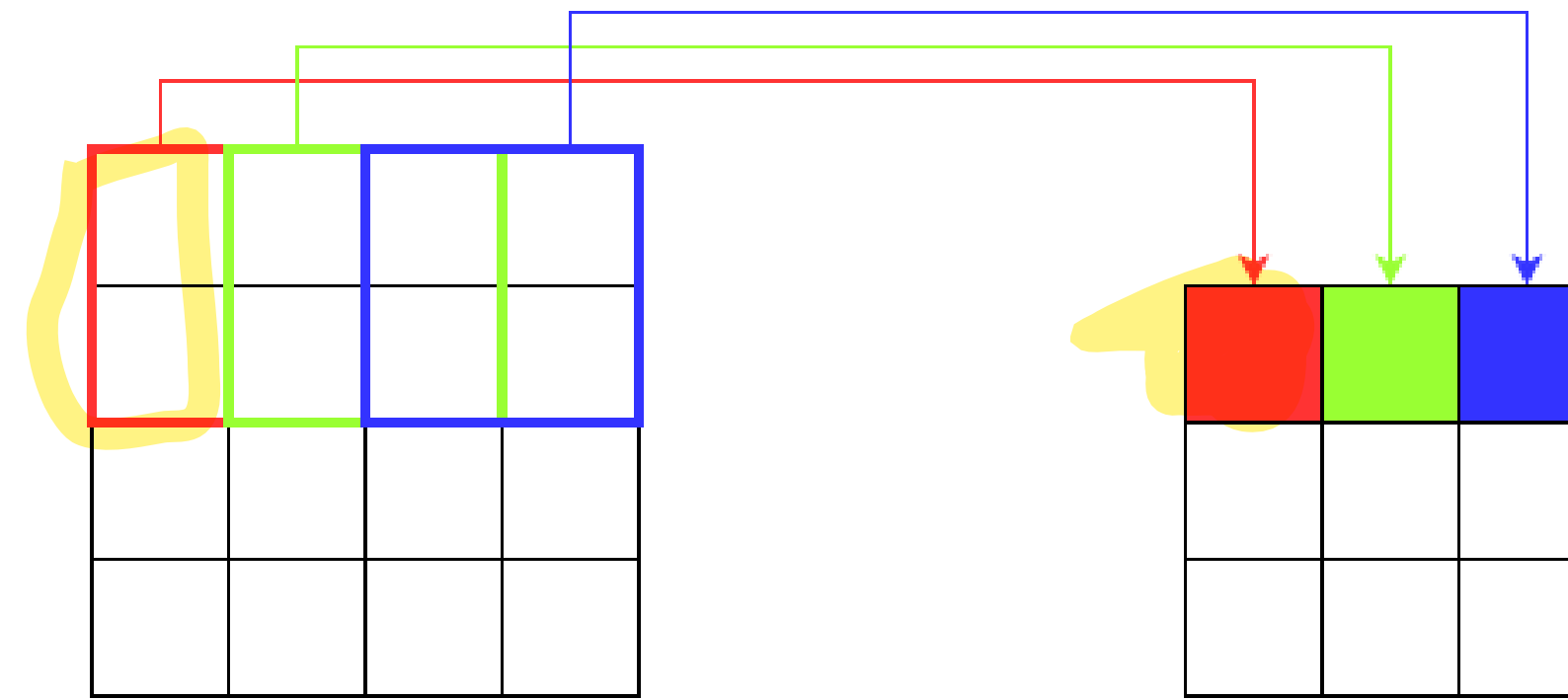
11	-14	4	5	8
34	5	9	-15	27
32	6	18	-3	13
4	29	-14	18	20
-1	5	9	13	15



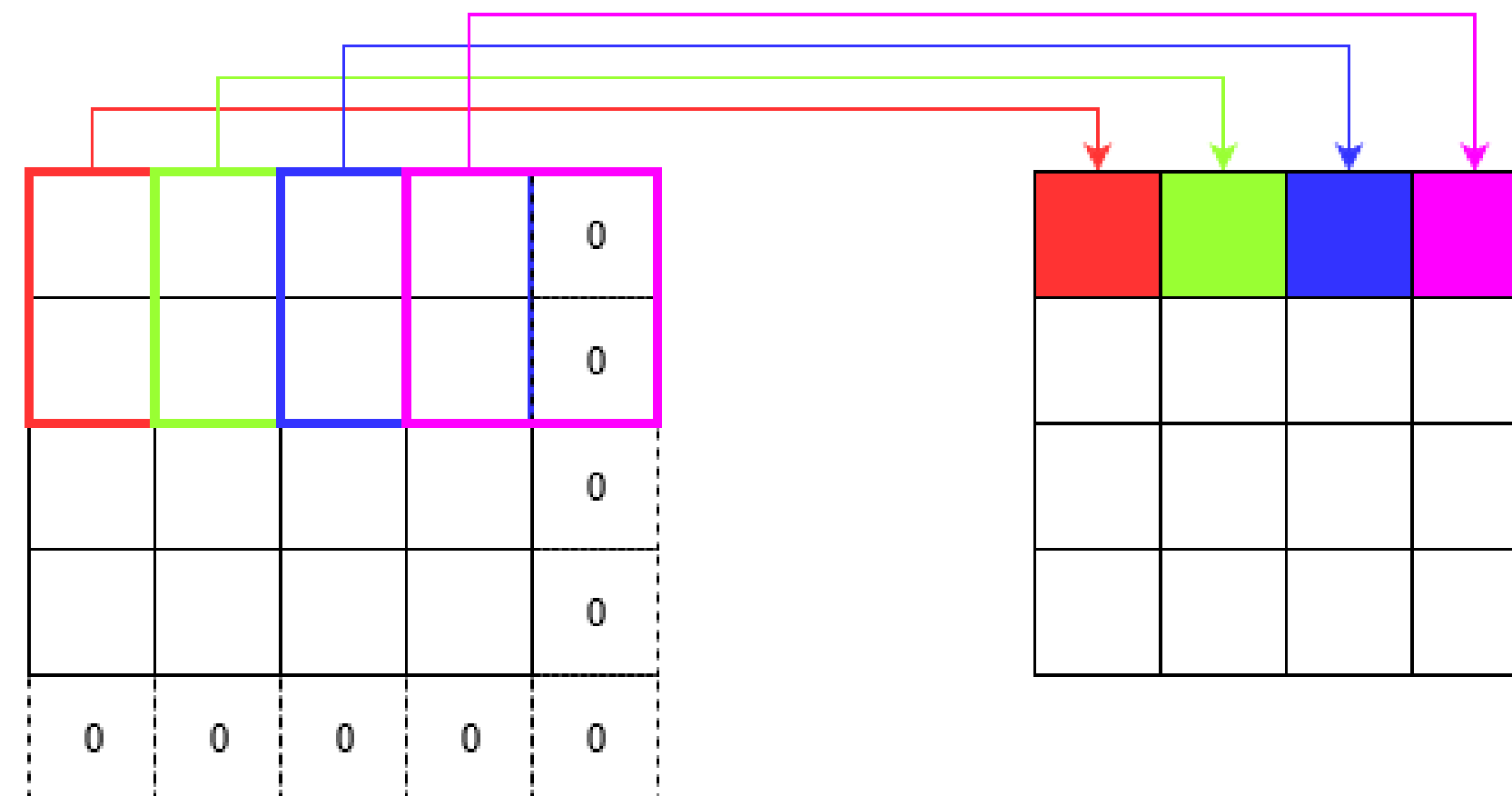
11	0	4	5	8
34	5	9	0	27
32	6	18	0	13
4	29	0	18	20
0	5	9	13	15

Padding

Padding=VALID

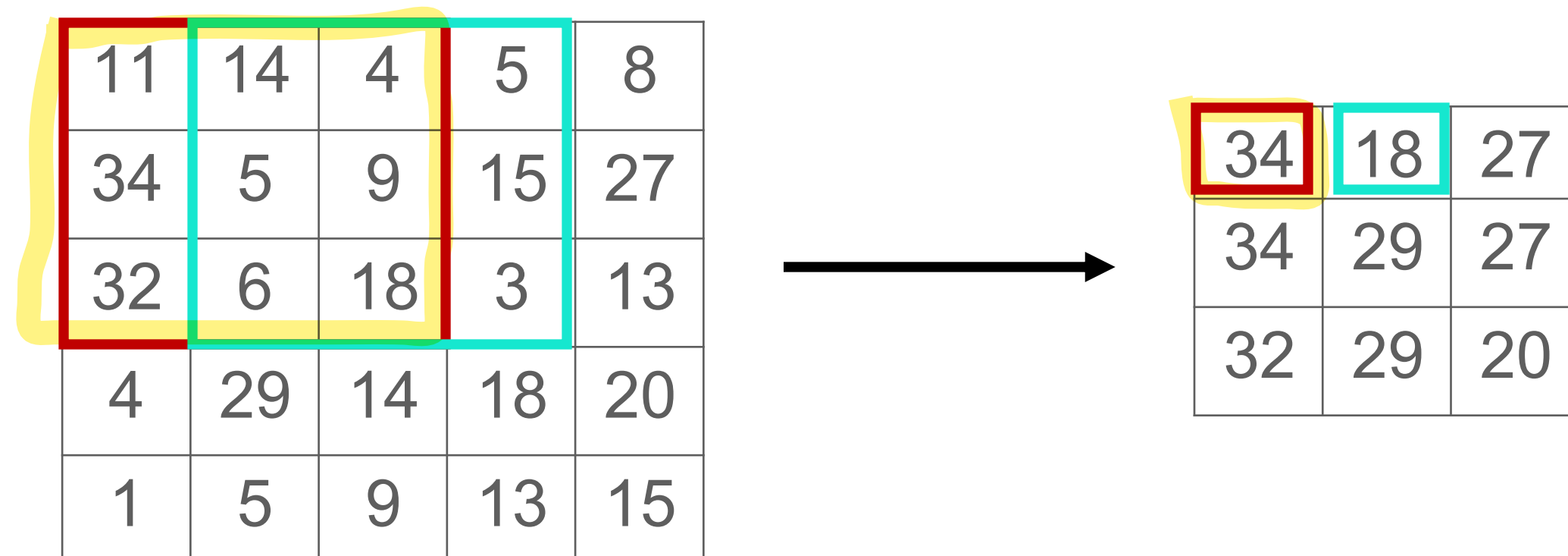


Padding=SAME



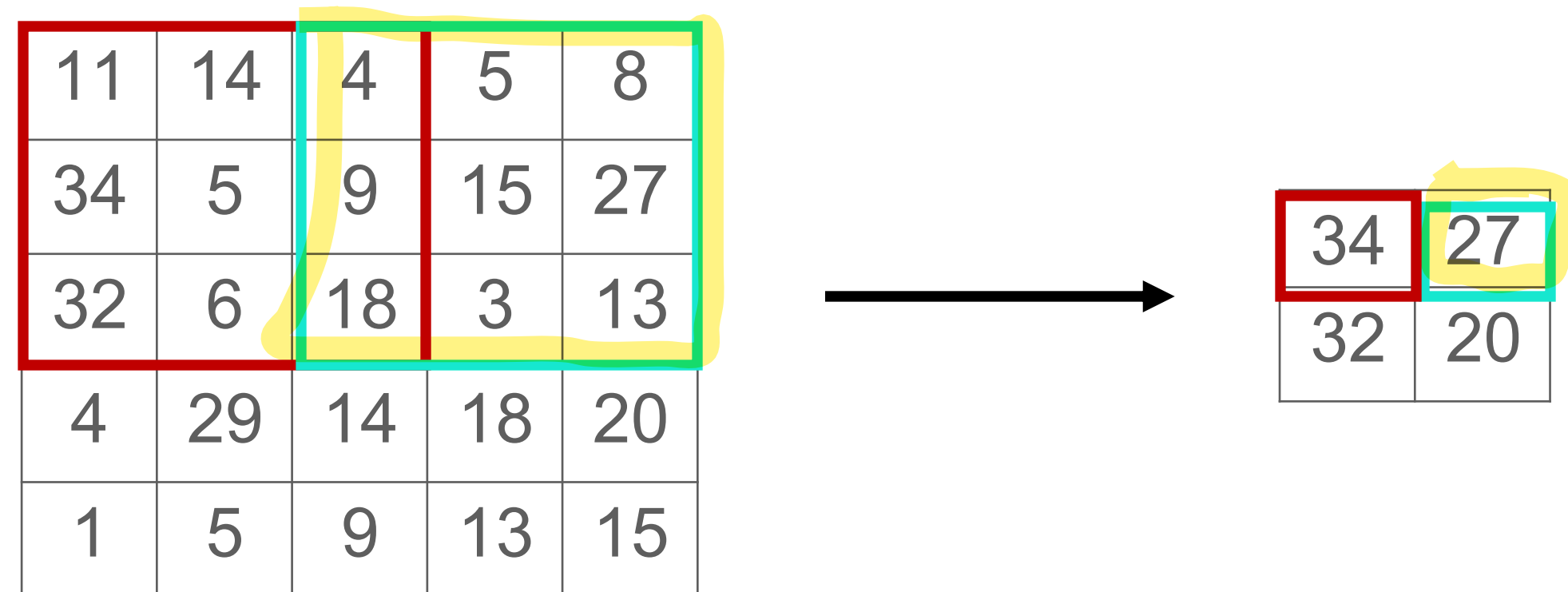
Pooling

- Pooling takes a feature map as input and produces another feature map.
- It summarises various regions on the feature map and by doing so it reduces the size/resolution of the feature map.
- Apply max pooling with pool size 3x3, with strides 1
 - What is the size of the resulting feature map?



Pooling

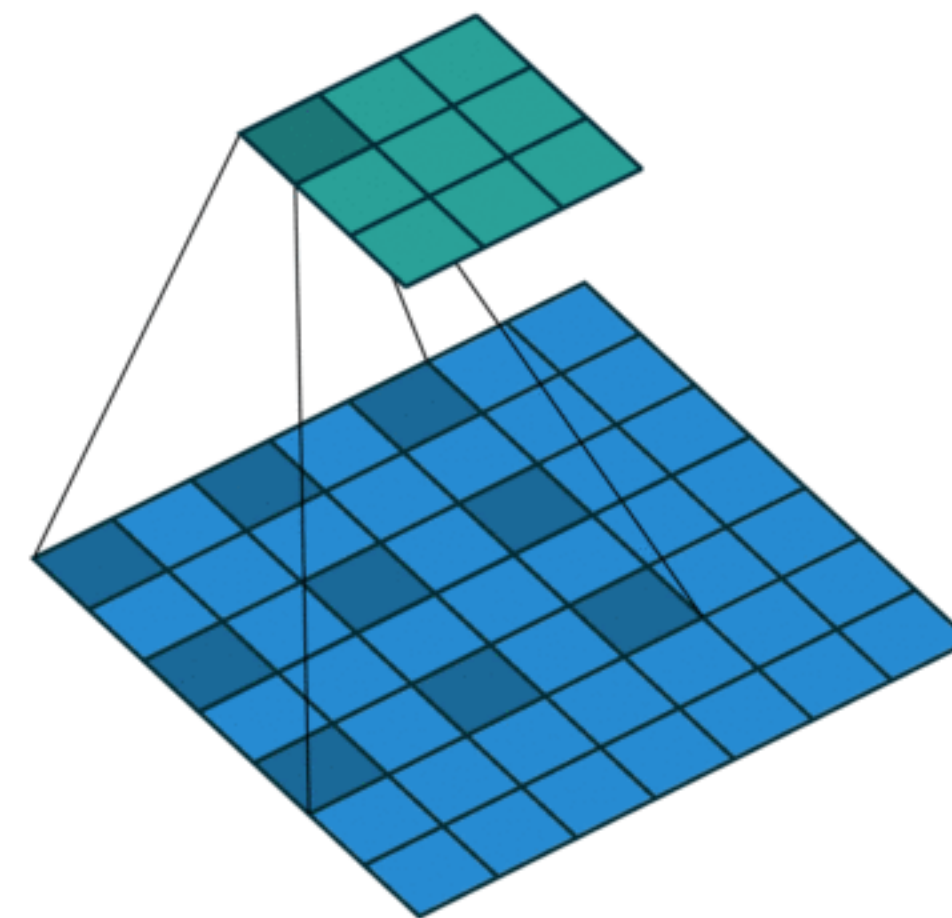
- Apply max pooling with pool size 3x3, with strides 2
- What is the size of the resulting feature map?



Other types of pooling:
average pooling, sum
pooling

Dilation rate

- Use dilation rate to give a wider field of view at the same computational cost.
- A 3x3 kernel (green) with a dilation rate of 2 will have the same field of view as a 5x5 kernel.

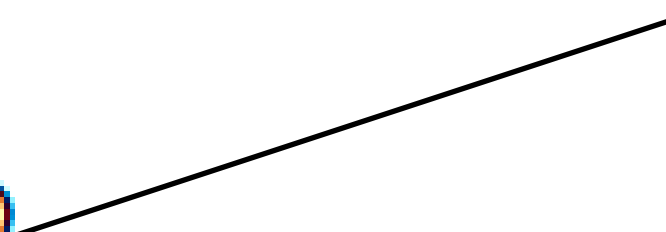


[source](#)

Example in Keras

```
model = Sequential()  
model.add(Conv2D(32, kernel_size=(5, 5), strides=(1, 1),  
                 activation='relu',  
                 input_shape=input_shape))  
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))  
model.add(Conv2D(64, (5, 5), activation='relu'))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Flatten())  
model.add(Dense(1000, activation='relu'))  
model.add(Dense(num_classes, activation='softmax'))
```

32 filters

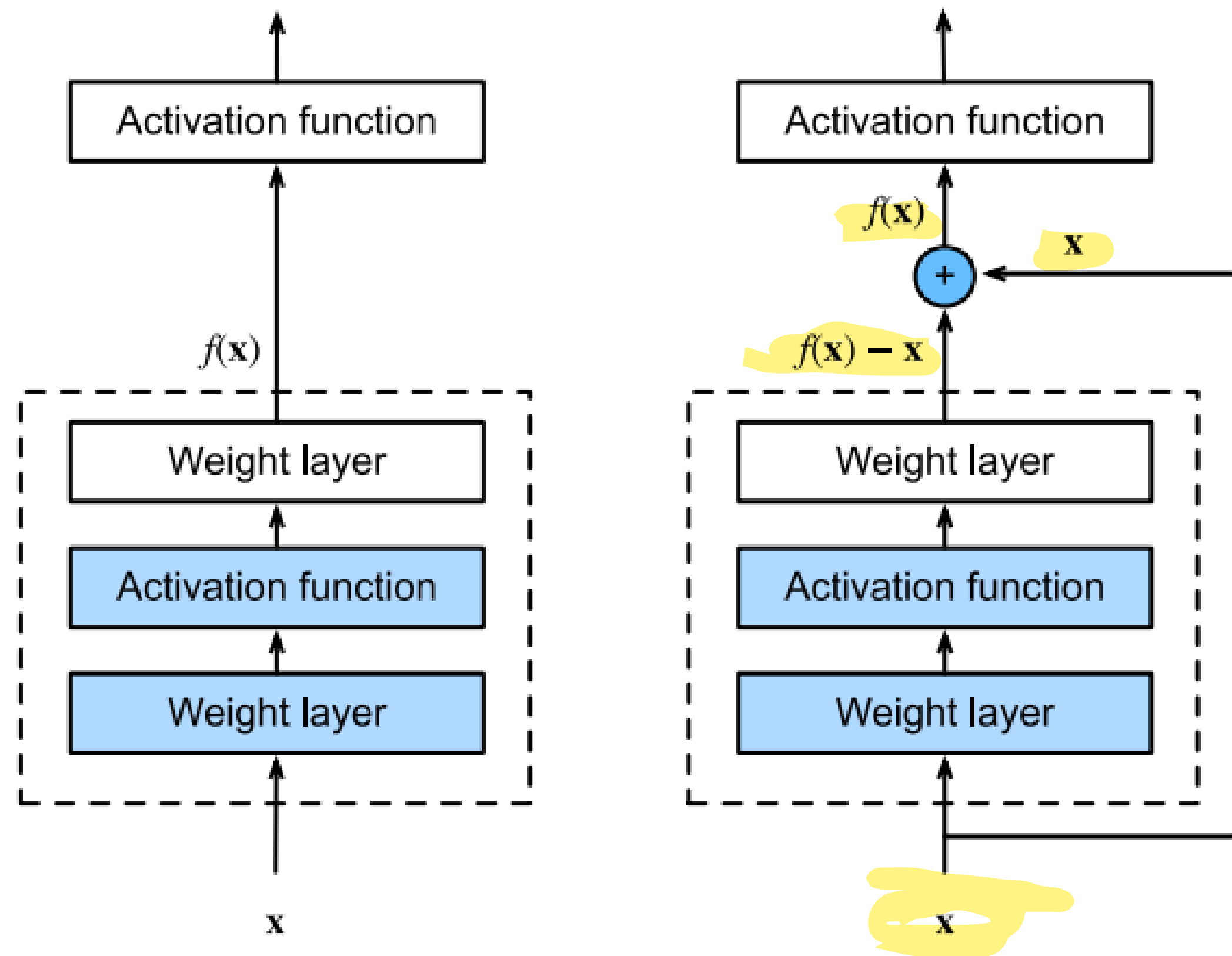


Hidden units

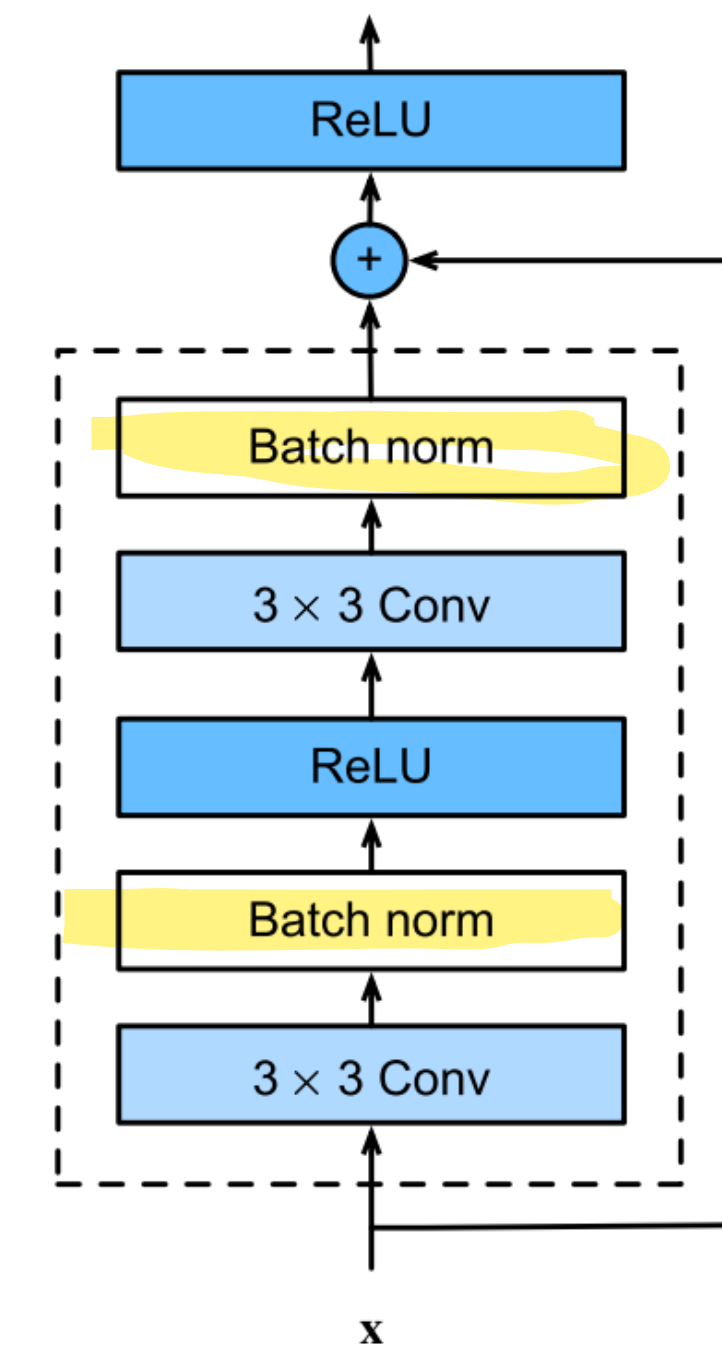


To allow training deep networks

Use residual connections and apply batch normalisation

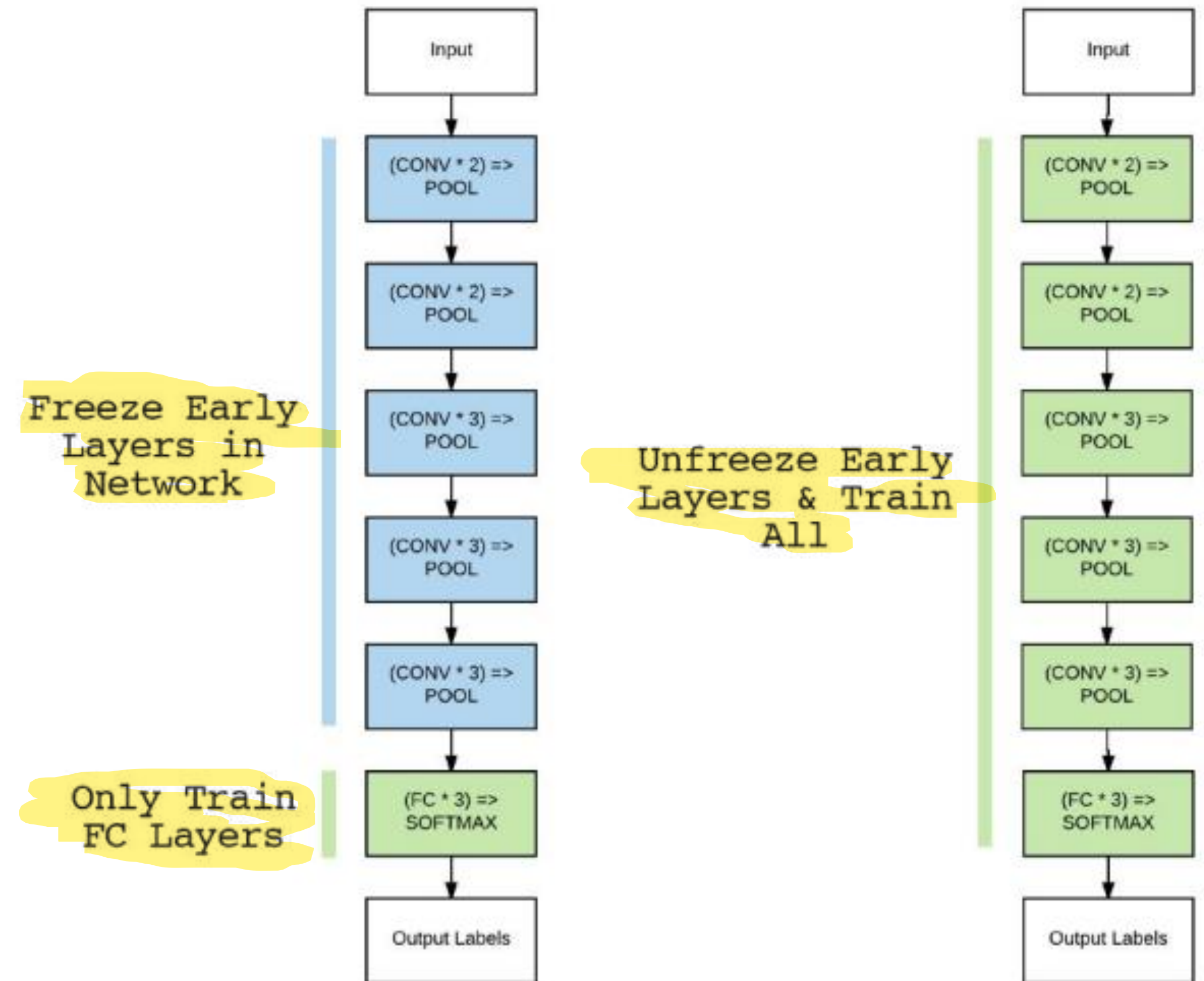


$$BN(x) = \gamma \frac{x - \mu_B}{\sigma_B} + \beta$$



Transfer learning

- Is a way of using previously learned knowledge to solve new task(s), by reusing part of a trained model.
- The main motivation is to reduce the need for training data.
- Advantages: reduce training time, re-use of useful knowledge
- Two approaches could be considered:
 - "freezing" (only replacing the top layer and retrain on the new task)
 - "fine-tuning" (modifying the pre-trained architecture).



Dealing with overfitting when using CNN

- L2 regularisation
- Early stopping
- Dropout
- Data augmentation for image-based task

```
tf.keras.preprocessing.image.ImageDataGenerator(  
    featurewise_center=False, samplewise_center=False,  
    featurewise_std_normalization=False, samplewise_std_normalization=False,  
    zca_whitening=False, zca_epsilon=1e-06, rotation_range=0, width_shift_range=0.0,  
    height_shift_range=0.0, brightness_range=None, shear_range=0.0, zoom_range=0.0,  
    channel_shift_range=0.0, fill_mode='nearest', cval=0.0,  
    horizontal_flip=False, vertical_flip=False, rescale=None,  
    preprocessing_function=None, data_format=None, validation_split=0.0, dtype=None  
)
```